

Lab 02 — Control Flow, Functions, Errors and Modules

DSA27 Lab Manual · Week 2 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 2

Contents

Part 1 — if statements	2
Part 2 — for loops and range	3
Part 3 — break, continue, else, pass	6
Part 4 — match statements	7
Part 5 — Defining functions	8
Part 6 — Errors and exceptions	13
Part 7 — Modules	17
Part 8 — Counting and timing: a first look at complexity	20
Part 9 — Exercises	21
Part 10 — Take-home practice (not graded)	22
Summary	22
Answers to the checkpoints	23

How to use this lab. As in Lab 01: type every example, predict before you run at each **Checkpoint** (answers at the end), then make `pytest tests/test_Lab02.py` pass. This lab assumes Lab 01 — if slicing or `b = a` still surprises you, go back first.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>labs/lab02.py</code> — 13 functions
Graded by	<code>tests/test_lab02.py</code>
Connects to	Lecture 02 — Complexity. Part 7 counts and times loops.

What you will be able to do

1. choose between `if/elif/else` and `match`, and write both;
 2. loop with `for` over any sequence, use `range`, `enumerate` and `zip`, and control a loop with `break`, `continue` and `else`;
 3. define functions with default, keyword, `*args` and `**kwargs` parameters, and avoid the mutable-default trap;
 4. explain what happens to a list when you pass it to a function;
 5. read a traceback, catch an exception with `try/except`, and raise your own;
 6. write a module, import it, and use the `if __name__ == "__main__":` guard;
 7. count the steps a loop takes and time it — the start of Week 2.
-

Part 1 — `if` statements

```
x = int(input("Please enter an integer: "))
if x < 0:
    x = 0
    print('Negative changed to zero')
elif x == 0:
    print('Zero')
elif x == 1:
    print('Single')
else:
    print('More')
```

There can be any number of `elif` (“else if”) parts, and the `else` is optional. Python checks the conditions **top to bottom** and runs the **first** block whose condition is true — then skips the rest. So order matters:

```
score = 95
if score >= 50:
    grade = 'pass'
elif score >= 90:           # never reached: 95 already matched above
    grade = 'excellent'
```

Put the most specific condition first.

1.1 What counts as **true**

Any value can be used as a condition. These are **false**:

`False`, `None`, `0`, `0.0`, `''` (empty string), `[]`, `()`, `{}`, `set()`, `range(0)`

Everything else is **true**. So the idiomatic way to test “is this list empty?” is:

```
if not values:
    print('empty')
```

rather than `if len(values) == 0:`.

1.2 Comparisons and Boolean operators

- Comparisons **chain**: `0 <= x < 10` means `0 <= x` and `x < 10`.
- and, or, not. not binds tightest, then and, then or: `a or b and not c` means `a or (b and (not c))`.
- and and or **short-circuit**: they stop as soon as the answer is known, and they return the **last value they looked at**, not necessarily True or False:

```
>>> 0 or 'default'
'default'
>>> 'x' and 'y'
'y'
>>> [] and 1/0          # 1/0 is never evaluated
[]
```

Short-circuiting is how you write a safe test:

```
if i < len(values) and values[i] == target:    # never an IndexError
    ...
```

- in and not in test membership: `'a' in 'banana'`, `3 in [1, 2, 3]`.
- is tests identity. Use it for None, and **only** for None (and True/False):

```
if result is None:          # correct
if result == None:          # works, but not the Python way
```

Checkpoint 1. What does each expression give?

- (a) `1 < 2 < 3 > 2` (b) `' '` or `[]` or `0` (c) `not []`
 (b) `5 and 0 and 1/0` (e) `[0]` and `'yes'`

Part 2 — for loops and range

2.1 for iterates over the items

Python's for is not C's counting loop. It walks through **the items** of any sequence, in order:

```
>>> words = ['cat', 'window', 'defenestrate']
>>> for w in words:
...     print(w, len(w))
...
cat 3
```

```
window 6
defenestrate 12
```

It works on anything iterable — strings, lists, tuples, dictionaries, files, ranges:

```
>>> for ch in 'DSA':
...     print(ch, end=' ')
...
D S A
```

Compare with the Lab 01 while version. There is no index to forget to increment, so there is no bug to write.

2.2 Do not change a collection while you loop over it

Removing items from a list while a for loop walks through it skips elements. Loop over a **copy**, or — better — build a new list:

```
users = ['ali', 'mona', 'guest', 'guest2', 'omar']

# wrong: skips 'guest2', because the list shifts under the loop
for u in users:
    if u.startswith('guest'):
        users.remove(u)

# right: loop over a copy
for u in users[:]:
    if u.startswith('guest'):
        users.remove(u)

# best: build a new list
active = []
for u in users:
    if not u.startswith('guest'):
        active.append(u)
```

2.3 range

To loop over numbers, use range:

```
>>> for i in range(5):
...     print(i, end=' ')
...
0 1 2 3 4
```

range(stop) gives 0, 1, ..., stop - 1 — “up to but not including”, exactly like slicing. range(n) has exactly n numbers in it. There are two more forms:

```

>>> list(range(5, 10))           # start, stop
[5, 6, 7, 8, 9]
>>> list(range(0, 10, 3))        # start, stop, step
[0, 3, 6, 9]
>>> list(range(-10, -100, -30)) # counting down
[-10, -40, -70]
>>> list(range(10, 0, -1))
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
>>> sum(range(4))                # 0 + 1 + 2 + 3
6

```

`range(10)` is **not a list**. It is a small object that produces the numbers on demand, so `range(10**12)` costs no memory. `list(range(...))` makes the list when you really want one.

2.4 enumerate and zip

When you need the index **and** the item, do not write `for i in range(len(values))`. Use `enumerate`:

```

>>> for i, name in enumerate(['ali', 'mona', 'omar']):
...     print(i, name)
...
0 ali
1 mona
2 omar
>>> for rank, name in enumerate(['ali', 'mona'], start=1):
...     print(f'{rank}. {name}')
...
1. ali
2. mona

```

To walk two sequences side by side, use `zip`. It stops at the shorter one:

```

>>> questions = ['name', 'quest', 'favourite colour']
>>> answers = ['Lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
...     print(f'What is your {q}? It is {a}.')
...
What is your name? It is Lancelot.
What is your quest? It is the holy grail.
What is your favourite colour? It is blue.

```

`range(len(values))` still has its place — when you need to compare `values[i]` with `values[i + 1]`, or write into `values[i]`. Sorting algorithms in Week 9 are full of it.

2.5 Nested loops

A loop inside a loop runs its inner body $\text{outer} \times \text{inner}$ times:

```
>>> for i in range(1, 4):
...     for j in range(1, 4):
...         print(f'{i * j:3}', end='')
...     print()
...
1  2  3
2  4  6
3  6  9
```

Remember that multiplication. It is where $O(n^2)$ comes from.

Part 3 — break, continue, else, pass

break leaves the innermost loop immediately. **continue** skips the rest of this iteration and goes to the next one.

```
>>> for num in range(2, 10):
...     if num % 2 == 0:
...         print(f'Found an even number {num}')
...         continue
...     print(f'Found an odd number {num}')
...
Found an even number 2
Found an odd number 3
Found an even number 4
Found an odd number 5
Found an even number 6
Found an odd number 7
Found an even number 8
Found an odd number 9
```

else on a loop runs when the loop finishes **without** hitting break. Think of it as “no break”. It is exactly what a search needs:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(n, 'equals', x, '*', n // x)
...             break
...     else:
...         # the inner loop found no factor
...         print(n, 'is a prime number')
```

```
...
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

Look carefully: the `else` belongs to the inner **for**, not to the `if`.

pass does nothing. Use it where Python needs a statement and you have none yet:

```
def todo():
    pass          # write this later

class EmptyForNow:
    pass
```

(`raise NotImplementedError`, as in the exercise files, is better for a function that must not be used before it is written — it fails loudly instead of silently returning `None`.)

Part 4 — match statements

`match` (Python 3.10+) compares a value against **patterns** and runs the first one that fits. At its simplest it replaces a chain of `elif ==`:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 418:
            return "I'm a teapot"
        case 401 | 403:          # | means "or"
            return "Not allowed"
        case _:                 # _ matches anything: the default
            return "Something's wrong with the internet"
```

Its real power is **destructuring**: a pattern can look like the data and pull it apart. Here the value is a list of words:

```
def run(command):
    match command.split():
        case ['quit']:
```

```

    return 'bye'
case ['go', direction]:           # exactly two words, binds direction
    return f'going {direction}'
case ['drop', *objects]:         # 'drop' then any number of words
    return f'dropping {len(objects)} things'
case ['go', ('north' | 'south')]: # never reached – see below
    return 'unreachable'
case _:
    return f'unknown command: {command!r}'

```

```

>>> run('go north')
'going north'
>>> run('drop sword shield')
'dropping 2 things'
>>> run('dance')
"unknown command: 'dance'"

```

- A plain name in a pattern (direction) **captures** whatever is there.
- `*objects` captures “the rest” as a list, possibly empty.
- Cases are tried **in order**; the fourth case above can never match because the second catches every two-word go.
- A **guard** adds a condition: `case ['go', d] if d in exits:`.

`{command!r}` in the f-string means “use `repr()`”, which shows quotes around a string — useful in error messages.

Part 5 — Defining functions

5.1 The basics

```

def fib(n):
    """Print the Fibonacci numbers less than n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a + b
    print()

>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597

```

- The first statement may be a **docstring**. Tools show it: try `help(fib)`. Write one for every function — in this course the docstring is the contract.
- A function is an object like any other. `f = fib` gives it a second name; `f(100)` then works.
- A function with no return, or a bare return, returns **None**:

```
>>> print(fib(0))
```

```
None
```

Better: **return** the values and let the caller decide what to do with them.

```
def fib2(n):
    """Return a List of the Fibonacci numbers less than n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a + b
    return result

>>> fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

5.2 Scope: local and global

Variables assigned inside a function are **local**: they exist only while the function runs, and they hide any global variable with the same name.

```
count = 0

def increment():
    count = count + 1      # UnboundLocalError: count is local here

def increment_ok():
    global count           # possible, but almost always a bad design
    count = count + 1
```

When a name is **read**, Python looks in this order: **L**ocal, **E**nclosing function, **G**lobal (the module), **B**uilt-in — the *LEGB* rule. Assigning to a name anywhere in a function makes it local for the whole function.

The rule for this course: a function gets what it needs from its **parameters** and gives results back with **return**. No globals.

5.3 What happens when you pass a list

Arguments are passed **by object reference** — the parameter becomes a new *name* for the *same object*. (This is the `b = a` rule from Lab 01 again.)

```
def add_item(values):
    values.append('new')    # changes the caller's list

def replace(values):
```

```

    values = ['brand', 'new']    # rebinds the LOCAL name only

>>> data = ['old']
>>> add_item(data)
>>> data
['old', 'new']
>>> replace(data)
>>> data
['old', 'new']

```

A function can **mutate** an object it is given, but it cannot make the caller's *name* point somewhere else. Numbers and strings are immutable, so a function can never change the caller's number or string.

This is why the exercises say “return a NEW list” or “the input must not change”. A test checks it.

5.4 Default argument values

```

def ask_ok(prompt, retries=4, reminder='Please try again!'):
    ...

ask_ok('Really quit?')
ask_ok('OK to overwrite?', 2)
ask_ok('OK to overwrite?', 2, 'Come on, only yes or no!')

```

The trap. A default value is evaluated **once**, when the `def` runs — not on every call. With a mutable default, every call shares one object:

```

def f(a, L=[]):
    L.append(a)
    return L

>>> f(1)
[1]
>>> f(2)
[1, 2]          # the SAME list as last time
>>> f(3)
[1, 2, 3]

```

The fix is to default to `None` and create the object inside:

```

def f(a, L=None):
    if L is None:
        L = []
    L.append(a)
    return L

```

This is exercise 11.

5.5 Keyword arguments

Arguments can be passed **by name**, in any order, after the positional ones:

```
def parrot(voltage, state='a stiff', action='vroom', type='Norwegian Blue'):
    print(f"-- This parrot wouldn't {action} if you put {voltage} volts through it.")
    print(f"-- Lovely plumage, the {type}. It's {state}!")

parrot(1000)                                # 1 positional
parrot(voltage=1000)                        # 1 keyword
parrot(action='VOOOOOM', voltage=1000000)   # 2 keywords, any order
parrot('a million', 'bereft of life', 'jump') # 3 positional
parrot(voltage=5.0, 'dead')                 # SyntaxError: positional after keyword
parrot(110, voltage=220)                     # TypeError: two values for voltage
```

Keyword arguments make calls readable: `sorted(names, reverse=True)` says what it means; `sorted(names, None, True)` would not.

Special markers in the parameter list control how arguments may be passed:

```
def f(pos_only, /, normal, *, kw_only):
    ...

f(1, 2, kw_only=3)           # fine
f(1, normal=2, kw_only=3)    # fine
f(pos_only=1, normal=2, kw_only=3) # TypeError
f(1, 2, 3)                   # TypeError: kw_only must be named
```

Everything before `/` is positional-only; everything after `*` is keyword-only. You will read these in library documentation more often than you write them.

5.6 Any number of arguments: `*args` and `**kwargs`

A parameter written `*name` collects **extra positional arguments** into a tuple. A parameter written `**name` collects **extra keyword arguments** into a dictionary.

```
def report(title, *values, **options):
    print(title, values, options)

>>> report('marks', 90, 75, 88, sep=';', rounded=True)
marks (90, 75, 88) {'sep': ';', 'rounded': True}
>>> report('none')
none () {}
```

The reverse — **unpacking** a list or dictionary into arguments at a call — uses the same stars:

```
>>> args = [3, 6]
>>> list(range(*args))           # same as range(3, 6)
[3, 4, 5]
>>> opts = {'sep': '-', 'end': '!\n'}
```

```
>>> print('a', 'b', **opts)      # same as print('a', 'b', sep='-', end='!\n')
a-b!
```

Exercise 12, `stats(*numbers)`, uses this.

5.7 lambda: small anonymous functions

`lambda args: expression` makes a function in one line. It is limited to a single expression, which it returns.

```
>>> square = lambda x: x * x
>>> square(7)
49
>>> pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
>>> pairs.sort(key=lambda pair: pair[1])      # sort by the word
>>> pairs
[(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]
```

`key=` functions are where you will use `lambda` most — for sorting, `min`, `max`.

Functions that return functions. A function defined inside another remembers the variables around it, even after the outer function has returned. That is called a *closure*:

```
def make_incrementor(n):
    return lambda x: x + n

>>> add42 = make_incrementor(42)
>>> add42(0)
42
>>> add42(1)
43
```

Exercises 12 and 13 practise passing and returning functions.

5.8 Docstrings, annotations and style

Docstring conventions. First line: a short summary sentence. If there is more, a blank line, then detail. Every `dsa/` skeleton follows this shape.

Annotations are optional hints about types. Python does not enforce them; editors and tools use them:

```
def greet(name: str, times: int = 1) -> str:
    return ('Hello ' + name + '! ') * times
```

Style (PEP 8), the parts that matter most:

- 4 spaces per indentation level, never tabs.
- Lines no longer than about 79–88 characters (the repo sets a ruler at 88).
- Blank lines between functions; blank lines inside functions sparingly.

- Comments on their own line when possible; a space after #.
- Spaces around operators and after commas: `a = f(1, 2) + g(3, 4)`.
- `lower_case_with_underscores` for functions and variables; `UpperCamelCase` for classes; `self` as the first method parameter.
- ASCII or UTF-8 only, and English names for code identifiers.

Readable code is not decoration. You will read your code far more often than you write it — and so will the TA marking it.

Checkpoint 2. *What is printed?*

```
def g(x, items=[]):
    items.append(x)
    x = x * 10
    return len(items)

n = 1
print(g(n), g(n), n)
```

Part 6 — Errors and exceptions

6.1 Two kinds of error

A **syntax error** means Python could not even read your code. Nothing runs:

```
>>> while True print('Hello world')
File "<stdin>", line 1
    while True print('Hello world')
            ^^^^^
SyntaxError: invalid syntax
```

The caret points at where Python noticed the problem. The real mistake is often **just before** it — here, the missing `:`.

An **exception** happens while the code runs:

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
NameError: name 'spam' is not defined
>>> '2' + 2
TypeError: can only concatenate str (not "int") to str
```

6.2 Reading a traceback

Put this in a file `trace_demo.py` and run it:

```
def average(values):
    return total(values) / len(values)

def total(values):
    result = 0
    for v in values:
        result += v
    return result

print(average([1, 2, 3]))
print(average([]))
```

```
2.0
Traceback (most recent call last):
  File "trace_demo.py", line 11, in <module>
    print(average([]))
    ~~~~~^
  File "trace_demo.py", line 2, in average
    return total(values) / len(values)
           ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

Read it **from the bottom up**:

1. **Last line:** the exception type and message — *what* went wrong.
2. **The frame above it:** the line that raised it — *where*, file and line.
3. **Further up:** who called that function, and who called that — *how we got there*. The top is where the program started.

That list of frames is the **call stack**: each function call pushes a frame, each return pops one. You will study the call stack properly in Week 3, when functions call *themselves*, and again in Week 6, when you build a stack yourself.

The exceptions you will meet this term:

Exception	Typical cause
NameError	a misspelt variable, or used before assignment
TypeError	wrong type: '2' + 2, calling a non-function, wrong number of arguments
ValueError	right type, wrong value: <code>int('abc')</code> , <code>[1, 2].index(9)</code>
IndexError	list or string index out of range
KeyError	dictionary key not present

Exception	Typical cause
<code>AttributeError</code>	<code>None.append(1)</code> — usually a function that returned <code>None</code>
<code>ZeroDivisionError</code>	<code>x / 0</code> , <code>x // 0</code> , <code>x % 0</code>
<code>RecursionError</code>	a recursive function with no working base case (Week 3)
<code>NotImplementedError</code>	every skeleton in <code>dsa/</code> and <code>labs/</code> — until you write it
<code>StopIteration</code>	an iterator has no more items (Lab 03)

6.3 Handling exceptions: try / except

```
while True:
    try:
        x = int(input('Please enter a number: '))
        break
    except ValueError:
        print('Oops! That was no valid number. Try again...')
```

How it runs:

1. The try block runs.
2. If no exception happens, the except block is skipped.
3. If an exception happens, the rest of the try block is skipped. If its type matches the except, that block runs and the program carries on after the whole statement.
4. If it does not match, it travels up to the caller — and if nobody catches it, the program stops with a traceback.

The full form:

```
def read_ratio(a, b):
    try:
        result = int(a) / int(b)
    except ValueError as err:                # bind the exception object
        print('not a number:', err)
        return None
    except ZeroDivisionError:
        print('cannot divide by zero')
        return None
    else:                                    # runs only if the try did NOT raise
        print('ok')
        return result
    finally:                                # ALWAYS runs, even after return
        print('done')
```

```
>>> read_ratio('6', '3')
ok
done
2.0
>>> read_ratio('6', '0')
cannot divide by zero
done
```

- Several types in one clause: `except (TypeError, ValueError):`.
- `else` holds the code that should run only on success — keeping it out of the `try` means you do not accidentally catch its errors too.
- `finally` is for clean-up that must happen no matter what: closing a file, releasing a lock.

Catch narrowly. A bare `except:` or `except Exception:` catches *everything*, including your own bugs and misspelt names, and hides them. Catch the specific exception you expect, around the smallest block of code that can raise it.

6.4 Raising exceptions

A function that is given input it cannot handle should **say so**, loudly:

```
def factorial(n):
    if n < 0:
        raise ValueError(f'factorial is undefined for negative n, got {n}')
    result = 1
    for k in range(2, n + 1):
        result *= k
    return result
```

```
>>> factorial(-1)
Traceback (most recent call last):
...
ValueError: factorial is undefined for negative n, got -1
```

Many skeletons in `dsa/` say “Raises `ValueError` for ...”. The tests check it like this:

```
def test_factorial_rejects_negative():
    with pytest.raises(ValueError):
        factorial(-1)
```

Returning `None`, `-1` or an error string instead would **fail** that test — and, more importantly, would let a wrong value travel silently into the rest of a program.

A bare `raise` inside an `except` block re-raises the exception you caught, after you have done something with it (logged it, for example).

6.5 assert

`assert condition, message` raises `AssertionError` if the condition is false. It states something you believe must be true at that point:

```
def mean(values):
    assert len(values) > 0, 'mean of an empty list'
    return sum(values) / len(values)
```

Use `assert` to catch **your own** bugs while developing, not to validate user input (Python can be run with assertions switched off). `pytest` tests are written as plain `assert` statements — that is all a test is.

Checkpoint 3. What is printed?

```
def f(x):
    try:
        print('A')
        y = 10 // x
        print('B')
    except ZeroDivisionError:
        print('C')
        return -1
    else:
        print('D')
        return y
    finally:
        print('E')

print(f(0))
print(f(5))
```

Part 7 — Modules

7.1 A file is a module

Any `.py` file is a **module**; its name is the file name without `.py`. Create `fibonacci.py`:

```
"""Fibonacci numbers module."""

def fib(n):
    """Print the Fibonacci series up to n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a + b
```

```

    print()

def fib2(n):
    """Return the Fibonacci series up to n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a + b
    return result

```

In the REPL, started in the same folder:

```

>>> import fibo
>>> fibo.fib(100)
0 1 1 2 3 5 8 13 21 34 55 89
>>> fibo.fib2(10)
[0, 1, 1, 2, 3, 5, 8]
>>> fibo.__name__
'fibo'

```

The forms of import:

import fibo	<i># use as fibo.fib(...)</i>
import fibo as fb	<i># use as fb.fib(...)</i>
from fibo import fib, fib2	<i># use as fib(...) directly</i>
from fibo import fib as f	<i># rename on import</i>
from fibo import *	<i># everything – avoid: you can no longer tell where a name came from</i>

A module's top-level code runs **once**, the first time it is imported in a program. If you edit `fibo.py`, a running REPL keeps the old version — restart it.

7.2 Scripts and modules at once: `__name__`

When a file is **run** (`python fibo.py`), Python sets its `__name__` to `"__main__"`. When it is **imported**, `__name__` is the module name. So this block runs only when the file is run directly:

```

if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))

```

```

python fibo.py 50
0 1 1 2 3 5 8 13 21 34

```

Importing `fibo` does not run it. Use this guard for demos and quick checks at the bottom of your own files.

7.3 Packages, and how `from dsa.recursion import factorial` works

A **package** is a folder of modules with an `__init__.py` file in it. The course repository has three: `dsa/`, `viz/` and `labs/`. A dotted name walks into the folder:

```
from labs.lab02 import is_prime      # Labs/Lab02.py, function is_prime
from dsa.recursion import factorial  # dsa/recursion.py
from viz.draw import draw_array     # viz/draw.py
```

Python finds `labs` because it searches the folders in `sys.path`, and the first of those is the folder you started Python in (or the script's folder). That is why you run everything **from the DSA27 folder**. `tests/conftest.py` adds the repository root to `sys.path` so `pytest` can always find the packages.

`dir(module)` lists the names a module defines — useful for exploring:

```
>>> import math
>>> dir(math)[:6]
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos']
```

7.4 The standard library: three modules you need now

Python comes with hundreds of modules. Three are enough for this month:

```
>>> import math
>>> math.sqrt(16), math.floor(-2.5), math.ceil(2.1), math.pi
(4.0, -3, 3, 3.141592653589793)
>>> math.log2(1024), math.isqrt(17)      # isqrt: integer square root
(10.0, 4)
>>> math.inf > 10**100                   # infinity, handy as a starting minimum
True

>>> import random
>>> random.seed(27)                      # same seed -> same "random" numbers every run
>>> random.randint(1, 6)                  # an int from 1 to 6 inclusive
>>> random.choice(['a', 'b', 'c'])
>>> random.sample(range(100), 5)          # 5 distinct values
>>> values = list(range(10)); random.shuffle(values) # in place

>>> import time
>>> start = time.perf_counter()           # a high-resolution clock, in seconds
>>> # ... the work you want to time ...
>>> elapsed = time.perf_counter() - start
```

Part 8 — Counting and timing: a first look at complexity

Lecture 02 says an algorithm's cost is how its work **grows** with the input. You can see that growth with two tools from this lab: a counter and a clock.

Count. Add a counter to the body of a loop and see how it depends on n :

```
def count_single(n):
    steps = 0
    for i in range(n):
        steps += 1
    return steps

def count_nested(n):
    steps = 0
    for i in range(n):
        for j in range(n):
            steps += 1
    return steps

for n in [10, 100, 1000]:
    print(n, count_single(n), count_nested(n))
```

```
10 10 100
100 100 10000
1000 1000 1000000
```

Multiply n by 10: the single loop does 10 times the work; the nested loop 100 times. That is $O(n)$ against $O(n^2)$, counted rather than claimed.

Time. Put this in `timing.py`:

```
import time

def is_prime_slow(n):
    """Try every divisor from 2 to n - 1."""
    if n < 2:
        return False
    for d in range(2, n):
        if n % d == 0:
            return False
    return True

for n in [1_000_003, 2_000_003, 4_000_037, 8_000_009]: # all prime
    start = time.perf_counter()
    is_prime_slow(n)
    print(f'{n:>10,} {time.perf_counter() - start:.3f} s')
```

Each n is about double the one before. The time should roughly double too — the signature of

$O(n)$. Now replace `range(2, n)` with a loop that stops when $d * d > n$ (exercise 5), and run it again. The times collapse, because the loop now runs about $\sqrt{n}$ times instead of n . For $n = 8,000,009$ that is under 3,000 iterations instead of 8 million.

Two warnings about timing, which Week 2 develops:

- A single measurement is noisy. Run it several times and keep the **smallest** (the other runs were interrupted by something else on your machine). `viz/complexity.py` does exactly this for you.
- Timing tells you about *this* machine and *this* input. Counting tells you the shape. You want both.

Part 9 — Exercises

All in `labs/lab02.py`. Run `pytest tests/test_lab02.py -v`.

#	Function	Practises
1	<code>classify_triangle(a, b, c)</code>	<code>if/elif</code> , chained <code>==</code> , <code>raise ValueError</code>
2	<code>course_result(coursework, final, attendance)</code>	order of conditions, range checks
3	<code>fizzbuzz(n)</code>	<code>for</code> , <code>range</code> , <code>%</code> , building a list
4	<code>is_prime(n)</code>	early return, stopping at $\sqrt{n}$
5	<code>primes_below(n)</code>	calling your own function
6	<code>first_repeated(values)</code>	nested loops, early exit
7	<code>count_pairs(n)</code>	counting a nested loop, then a formula
8	<code>calculator(command)</code>	match with sequence patterns
9	<code>parse_int(text, default=None)</code>	<code>try/except ValueError</code>
10	<code>stats(*numbers)</code>	<code>*args</code> , a loop that tracks min and max
11	<code>append_to(item, target=None)</code>	the mutable-default trap, <code>is None</code>
12	<code>make_multiplier(k)</code>	returning a function
13	<code>apply_n(f, x, n)</code>	passing a function

Exercise 2 applies the real rules of this course, from the course guide: you need at least 75% attendance to sit the final, at least 30% of the final, and at least 60% overall.

Hints:

- **1.** Validate first (positive sides, then the triangle inequality), classify second. `a == b == c` is a chained comparison.

- **4.** `d * d <= n` avoids floats; `math.isqrt(n)` is the other clean way. Check `n < 2` before anything else.
- **6.** For each position `i`, look for values[`i`] among the elements *before* it. The first `i` where you find it gives the answer.
- **7.** For `n = 4` the pairs are (0,1) (0,2) (0,3) (1,2) (1,3) (2,3). How many pairs can you make from `n` things?
- **8.** `match command.split()`: with cases like `case ['add', a, b]:`. The captured `a` and `b` are **strings**; convert with `int()`. What happens to `int('x')`? Is that the exception the docstring asks for?
- **10.** Start `smallest` and `largest` at `numbers[0]`, not at `0` — why not `0`?
- **12.** `return lambda x: x * k`, or an inner `def`.

Part 10 — Take-home practice (not graded)

1. Write `print_calendar(first_weekday, days)` that prints a month like a wall calendar, seven right-aligned columns, starting on the given weekday (0 = Saturday). Use `range`, `end=' '` and `%`.
2. The *digital root* of 942 is $9 + 4 + 2 = 15 \rightarrow 1 + 5 = 6$. Write `digital_root(n)` with loops only. Then find a one-line formula using `%` and check it against your loop for every `n` below 10,000.
3. Write `binary(n)` that returns the binary digits of a non-negative integer as a string, using `//` and `%`. Compare with `bin(n)`. How many loop iterations does it take for `n = 1,000,000`? Why that number?
4. Write a script `guess.py` that picks a random number from 1 to 100 and lets the user guess, saying “higher” or “lower” and handling non-numbers with `try/except`. With the best strategy, what is the largest number of guesses ever needed? (You have met this number twice now.)
5. Extend Part 8: plot, on paper, the times of `is_prime_slow` against `n`. Is it a straight line?

Summary

Idea	The one line to keep
<code>if/elif</code>	First true condition wins. Most specific first.
Truthiness	<code>0, '', [], {}, None</code> are false. <code>if not values:</code>
<code>for</code>	Iterates over items. <code>enumerate</code> for the index, <code>zip</code> for pairs.
<code>range(n)</code>	<code>0 .. n-1</code> , lazy, exactly <code>n</code> numbers.
Loop <code>else</code>	Runs when the loop ends without <code>break</code> .
<code>match</code>	Patterns that look like the data; <code>_</code> is the default.

Idea	The one line to keep
Arguments	Passed by object reference: mutations show, rebinding does not.
Defaults	Evaluated once. Never a mutable default: use <code>None</code> .
Exceptions	Read tracebacks bottom-up. Catch narrowly. <code>raise</code> on bad input.
Modules	A file is a module; a folder with <code>__init__.py</code> is a package.
Cost	Nested loops multiply. Count it, then time it.

Answers to the checkpoints

Checkpoint 1. (a) `True` — `1 < 2` and `2 < 3` and `3 > 2`. (b) `0` — every value is false, so `or` returns the last one. (c) `True` — an empty list is false. (d) `0` — `and` stops at the first false value, so `1/0` never runs. (e) `'yes'` — `[0]` is a **non-empty** list, so it is true.

Checkpoint 2. `1 2 1`. The default list is created once and shared, so the second call sees the item left by the first. `x = x * 10` rebinds the local name only, so `n` is still `1`.

Checkpoint 3.

```
A
C
E
-1
A
B
D
E
2
```

For `f(0)`: the division raises, so `B` is skipped; the `except` prints `C` and returns `-1` — but `finally` still runs, printing `E`, before the value reaches `print`. For `f(5)`: no exception, so `except` is skipped, `else` prints `D` and returns `2`, and `finally` prints `E` first.